Multi-Agent Cyber-Threat Detection and Response System Using CrewAI and RAG

Introduction to Artificial Intelligence (CSS_2203)
IV SEMESTER MINI PROJECT REPORT
Submitted
to

MANIPAL ACADEMY OF HIGHER EDUCATION

School of Computer Engineering

<Branch><Section>

Submitted by

<STUDENT NAME>

<reg no.>

<STUDENT NAME>

<reg no.>

<STUDENT NAME>

<reg no.>

Under the guidance of

Dr. Simi V.R.
Assistant Professor- Senior Scale
School of Computer Engineering
Manipal Institute of Technology, Bengaluru



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

April 2026

MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

ABSTRACT

Security teams are overwhelmed by alerts and spend much of their time on manual triage: mapping each alert to an attack technique, looking up the right response procedure, applying containment and writing the incident report. This project designs and implements a multi-agent system that automates this loop. Security logs from authentication, firewall and web sources are first analysed by a hybrid detector that combines signature and threshold rules with an Isolation Forest anomaly model. The resulting alerts are handled by a crew of four cooperating LLM agents built with CrewAI: a Log Analyst, a Threat Intelligence Analyst, an Incident Responder and a Report Writer.

The agents are grounded with Retrieval-Augmented Generation (RAG). A ChromaDB vector store indexes MITRE ATT&CK techniques, sample CVEs and incident-response playbooks, so that technique mapping and containment steps are retrieved from curated knowledge instead of being generated from memory. Response actions are simulated and restricted to an allow-list, and the system falls back to a deterministic offline mode when no LLM is available, which keeps every result reproducible.

The system was evaluated on labelled synthetic telemetry containing 2,500 benign events and six injected attacks (SSH brute force, port scan, SQL injection, C2 beaconing, data exfiltration and privilege escalation). All 196 attack events were detected and correctly classified (recall 1.00). Precision was 1.00 for the rule layer alone and 0.925 (F1 = 0.961) when the anomaly model was added. RAG retrieved the correct ATT&CK technique for 7 of 8 test queries, and 25 automated tests pass. The work is delivered as a Python package and a self-contained Jupyter/Colab notebook.

Keywords: Multi-agent systems, CrewAI, Retrieval-Augmented Generation, cyber-threat detection, incident response, MITRE ATT&CK, Isolation Forest, large language models

Table of Contents

		Page No
Abstract		ii
List Of Tables		iv
List Of Figures		v
Chapter 1	**INTRODUCTION**	1
1.1	Overview of the project work	1
1.2	Motivation and background of the study	1
1.3	Problem statement and relevance	2
1.4	Objectives	2
Chapter 2	**BACKGROUND THEORY / LITERATURE REVIEW**	3
2.1	Summary of existing studies / works	3
2.2	Comparative analysis of approaches	4
2.3	Key findings, trends, and research gaps	4
2.4	Conclusion	4
Chapter 3	**METHODOLOGY**	5
3.1	Methodology with diagrams	5
3.2	System requirements	8
3.3	Evaluation methodology	9
Chapter 4	**IMPLEMENTATION DETAILS AND RESULT ANALYSIS**	10
4.1	Implementation details	10
4.2	Results/observations	11
4.3	Analysis of the results	12
Chapter 5	**CONCLUSION AND FUTURE SCOPE**	14
5.1	Conclusion	14
5.2	Limitations	14
5.3	Suggestions for future directions	14
REFERENCES		15

LIST OF TABLES

Table No	Table Title	Page No
2.1	Comparison of approaches to threat detection and response	4
3.1	Injected attack scenarios (196 attack events in total)	6
3.2	Detection rules, severities and ATT&CK mapping	6
3.3	Knowledge-base corpus	7
3.4	Agents, goals and tools	8
3.5	Software, hardware and service requirements	8
4.1	Implementation modules	10
4.2	Event-level detection results (seed 42)	11
4.3	Per-attack detection and classification	11
4.4	RAG retrieval test queries (hashing embedder)	12
4.5	Simulated response actions generated for each incident	12

LIST OF FIGURES

Figure No	Figure Title	Page No
3.1	System architecture of the multi-agent detection and response pipeline	5
3.2	RAG indexing and retrieval pipeline	7
4.1	Injected attack events, alerts by severity and attack events over time	11
4.2	Precision, recall and F1-score of the two detector configurations	11

CHAPTER 1
INTRODUCTION

1.1 Overview of the project work

This project builds a Security Operations Centre (SOC) style assistant that automates the life-cycle of a cyber-incident: **detection, enrichment, response and reporting**. Logs from three sources (authentication, firewall and web server) are analysed by a hybrid detection layer that combines signature and threshold rules with an unsupervised Isolation Forest model. Every alert is then passed to a crew of four cooperating LLM agents implemented with CrewAI: a Log Analyst / Threat Detector, a Threat Intelligence Analyst, an Incident Responder and a Security Report Writer.

The agents do not rely only on what a language model remembers. They are grounded through **Retrieval-Augmented Generation (RAG)** over a knowledge base of MITRE ATT&CK techniques, CVE descriptions and incident-response playbooks, stored in a ChromaDB vector database. The Threat Intelligence Analyst retrieves the matching ATT&CK technique for an alert, and the Incident Responder retrieves the relevant playbook and turns its containment steps into actions such as blocking an IP address, isolating a host or disabling an account. These actions are simulated and are restricted to an allow-list.

The system is delivered as a Python package and as a single self-contained Jupyter/Google Colab notebook. It works with free LLM APIs (Google Gemini, Groq or OpenRouter) and also has an offline, deterministic mode that runs the same tools and RAG pipeline without any API key, so the results in this report can be reproduced by anyone. The project applies several topics of the course Introduction to Artificial Intelligence: intelligent agents and multi-agent cooperation, knowledge-based reasoning and retrieval, rule-based systems, and unsupervised machine learning for anomaly detection.

1.2 Motivation and background of the study

Modern organisations generate very large volumes of security telemetry. Security analysts must decide, for each alert, whether it is a real attack, which technique the attacker is using, what to do about it and how to document it. This work is repetitive, time-critical and depends on analysts remembering or searching through large bodies of reference material such as the MITRE ATT&CK matrix [7] and the organisation's response procedures. When alert volumes grow faster than the team, analysts suffer from alert fatigue, and response is delayed.

Two developments make automation of this work more realistic than before. First, large language models can read unstructured text, reason about it and call tools, and agent frameworks such as ReAct [4], AutoGen [5] and CrewAI [10] let several role-specific agents cooperate on a task. Second, Retrieval-Augmented Generation [2] allows a model to look up trusted documents at answer time, which reduces hallucination and lets the knowledge be

updated by editing documents instead of retraining a model. Combining detection algorithms, RAG and multi-agent orchestration gives a design in which each component does what it is best at: algorithms find events, retrieval supplies facts, and agents coordinate the reasoning and communication.

1.3 Problem statement and relevance

Problem statement: Design and implement a multi-agent system that, given heterogeneous security logs, (i) detects common attacks, (ii) maps each incident to the MITRE ATT&CK framework using retrieved knowledge, (iii) selects and applies containment steps from incident-response playbooks, and (iv) produces an incident report, while remaining auditable, safe to run and reproducible with free resources.

The problem is relevant because existing tools address only part of it. Rule-based and machine-learning detectors raise alerts but neither explain them nor propose a response. General-purpose language-model assistants can explain and advise but may invent details when they are not grounded in trusted sources. An end-to-end, grounded and testable pipeline therefore addresses a real gap between detection and response. The scope of this project is limited to a simulated environment: the logs are synthetic, the knowledge base is a curated subset, and no real network device is controlled.

1.4 Objectives

- To generate labelled, multi-source security telemetry containing six attack types so that detection can be measured objectively.
- To build a hybrid detector that combines explainable rules with an Isolation Forest anomaly model.
- To build a RAG knowledge base (ATT&CK, CVEs and response playbooks) in ChromaDB and to use it for threat enrichment and response planning.
- To orchestrate four specialised agents with custom tools using CrewAI, with support for free LLM providers.
- To keep response actions safe (simulated and allow-listed) and the system reproducible through an offline mode and automated tests.
- To evaluate detection quality, retrieval quality and response correctness, and to document the system and its limitations.

CHAPTER 2
BACKGROUND THEORY / LITERATURE REVIEW

2.1 Summary of existing studies / works

Rule-based and machine-learning detection

Traditional intrusion detection and SIEM correlation rely on signatures and thresholds, for example "more than N failed logins within M minutes". Rules are precise and explainable but only find what they describe. Machine-learning approaches complement them. Supervised classifiers are trained on labelled benchmark datasets such as CICIDS2017 [6], but need labelled data and tend to degrade when traffic changes. Unsupervised methods need no labels. The Isolation Forest of Liu, Ting and Zhou [1] isolates observations by random partitioning; anomalies need fewer partitions to be isolated, so the method scores outliers in roughly linear time and is well suited to behaviour that no rule anticipates.

Retrieval-Augmented Generation and embeddings

Lewis et al. [2] introduced RAG, which combines a generative model with a non-parametric memory that is searched at generation time. Retrieval gives the model access to specific, up-to-date documents and makes the source of an answer inspectable. Dense retrieval depends on sentence embeddings; Sentence-BERT [3] showed how to obtain semantically meaningful sentence vectors that can be compared with cosine similarity, and the compact all-MiniLM-L6-v2 model in that family is widely used for this purpose. Vector databases such as ChromaDB [11] store the vectors with metadata and perform nearest-neighbour search.

LLM agents and multi-agent frameworks

ReAct [4] showed that interleaving reasoning with tool calls lets a language model act on external information instead of only generating text. AutoGen [5] generalised this to conversations between several agents. CrewAI [10] provides a role-oriented abstraction in which each agent has a role, goal, backstory and a set of tools, and tasks are executed in a defined process (sequential or hierarchical) with the output of one task passed as context to the next. LiteLLM [14] gives CrewAI a common interface to many hosted models.

Security knowledge sources

MITRE ATT&CK [7], [12] is a curated knowledge base of adversary tactics and techniques and is the common vocabulary for describing attacker behaviour. NIST SP 800-61 [8] defines the incident-handling life-cycle (preparation, detection and analysis, containment, eradication and recovery, and post-incident activity) that the response playbooks in this project follow.

2.2 Comparative analysis of approaches

Table 2.1: Comparison of approaches to threat detection and response

Approach	Strengths	Weaknesses	Role in this project
Signature / rule-based detection	Precise, fast, fully explainable	Detects only known patterns; manual upkeep	First-line detector for six attack types
Unsupervised ML (Isolation Forest)	No labels needed; finds unexpected behaviour	Produces false positives; weak explanations	Second-line detector; low-severity tickets only
Supervised ML intrusion detection	High accuracy on matching data	Needs labelled data; poor transfer to new traffic	Not used (no real labelled data)
Standalone LLM assistant	Flexible reasoning and writing	Can hallucinate; knowledge frozen at training time	Not used alone
RAG-grounded LLM	Answers tied to trusted documents; easy to update	Quality depends on retrieval	Technique mapping and playbook retrieval
Multi-agent RAG (proposed)	Specialised roles, tool use, auditable steps	More moving parts; needs a capable LLM	Complete detect-enrich-respond-report loop

2.3 Key findings, trends, and research gaps

- Hybrid detection (explainable rules plus an anomaly model) is a common practical compromise between precision and coverage.
- Grounding language models with retrieval is the standard way to reduce hallucination and to keep knowledge current without retraining.
- Role-specialised agents with explicit tools are replacing single, monolithic prompts for multi-step workflows.
- **Gap 1:** detection tools usually stop at an alert, with no mapping to ATT&CK and no response guidance.
- **Gap 2:** few open, reproducible implementations connect detection, retrieval, agent orchestration and response in one pipeline that can be run without paid services.
- **Gap 3:** automated response needs safety controls (simulation, allow-lists, human review for low-confidence alerts), which are often missing from demonstrations.

2.4 Conclusion

The literature shows mature building blocks (Isolation Forest, dense retrieval, agent frameworks, ATT&CK and NIST guidance) but little integration of them into a small, testable and safe end-to-end system. This project fills that gap by combining these components in a reproducible pipeline and evaluating each stage.

CHAPTER 3
METHODOLOGY

3.1 Methodology with diagrams

3.1.1 Overall architecture

The system is a pipeline of four stages. Deterministic detection code first turns raw logs into alerts. A sequential CrewAI crew then enriches each alert, plans and applies the response, and writes the report. The RAG knowledge base is shared by the two agents that need reference knowledge. Figure 3.1 shows the architecture.

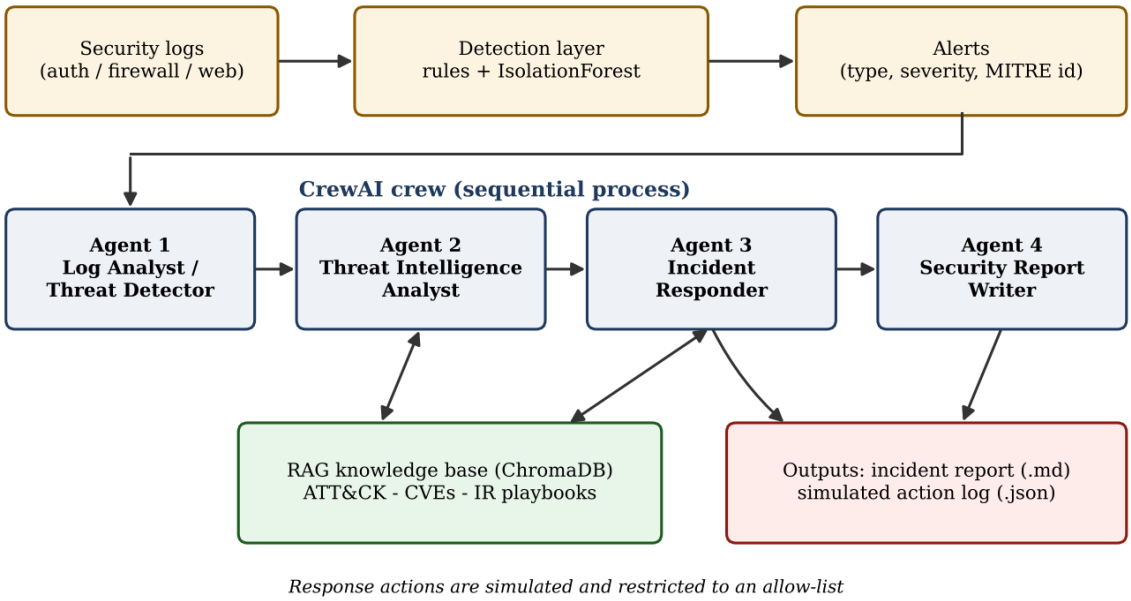


Figure 3.1: System architecture of the multi-agent detection and response pipeline

Detection is deliberately kept outside the language model. Rules and the anomaly model are cheap, reproducible and testable, while the model's tokens are spent on enrichment, response planning and communication. All agent tools call the same plain Python functions that the offline mode uses, so the offline pipeline exercises the real logic.

3.1.2 Synthetic telemetry

Real, labelled logs that contain a full attack life-cycle are hard to obtain and often cannot be shared, so a generator produces a three-hour log with a single schema (event id, timestamp, source, source and destination IP, destination port, host, user, action, bytes out, request, status and label). It adds 2,500 benign events (25% authentication, 40% web and 35% firewall) and injects one incident for each attack type, as listed in Table 3.1. The ground-truth label is used only for evaluation. A fixed random seed makes the data reproducible.

Table 3.1: Injected attack scenarios (196 attack events in total)

Attack	Pattern injected	Events
SSH brute force	70 failed logins from one external IP in about 105 s across 6 account names, followed by one successful login	71
Port scan	60 distinct destination ports probed on the database host, all denied by the firewall, 0.4 s apart	60
SQL injection	12 web requests carrying UNION SELECT, OR 1=1, DROP TABLE, SLEEP() and information_schema payloads	12
C2 beaconing	40 small (180-260 byte) connections from an internal host to an external IP on port 4444 every 60 s with about 1 s jitter	40
Data exfiltration	8 uploads of 25-60 MB each (352 MB in total) from the file server to a rare external IP	8
Privilege escalation	5 commands: sudo to a shell, chmod u+s, a NOPASSWD sudoers entry, usermod -aG sudo, su root	5

3.1.3 Detection layer

The rule layer contains one detector per attack type, summarised in Table 3.2. Each rule produces an alert with a type, severity, source IP, host, evidence text, MITRE technique identifier and the identifiers of the events it covers. For network-facing attacks the alert also records the remote malicious address (the attacker, or the external C2 or exfiltration destination), which the response stage later blocks.

Table 3.2: Detection rules, severities and ATT&CK mapping

Attack	Rule logic	MITRE	Severity
SSH brute force	10 or more failed logins from one IP within 5 minutes; a later success is flagged as possible compromise	T1110	High
Port scan	20 or more distinct denied destination ports from one IP	T1046	Medium
SQL injection	Regular expression for common SQL-injection payloads in web requests	T1190	High
C2 beaconing	15 or more connections to one external IP, coefficient of variation of the interval below 0.1, median payload below 5 KB	T1071	Critical
Data exfiltration	More than 100 MB sent to one external IP	T1041	Critical
Privilege escalation	Privilege-change events matching setuid, sudoers and sudo-group patterns	T1548	Critical
Anomaly (model)	Isolation Forest outlier not already explained by a rule	T1595	Low

The anomaly layer uses scikit-learn's IsolationForest (200 trees, contamination 0.002) on features computed for every (source IP, 5-minute window) pair: number of events, number of failed or blocked events, number of distinct ports, total bytes out and number of distinct targets, all log-transformed. Windows whose events are all explained by a rule alert are skipped. Anomaly alerts have Low severity and, by design, only open a ticket for an analyst and never trigger automatic containment.

3.1.4 RAG knowledge base

The knowledge base is stored as files so that it can be reviewed and extended without changing code (Table 3.3). Each ATT&CK technique and each CVE becomes one chunk; each playbook is split by section so that retrieval can return a specific step. Every chunk carries metadata (document type, MITRE identifier, title). In total 39 chunks are indexed: 10 techniques, 5 CVEs and 24 playbook sections.

Table 3.3: Knowledge-base corpus

File	Content	Chunking
mitre_attack_techniques.json	10 ATT&CK techniques: id, name, tactic, description, mitigation	1 chunk per technique
cve_samples.json	5 well-known CVEs mapped to techniques	1 chunk per CVE
playbooks/*.md	6 incident-response playbooks (indicators, containment, eradication/recovery, post-incident)	1 chunk per section
ioc_feed.json	Small simulated threat-intelligence feed of malicious IPs	Looked up directly, not embedded

Chunks are embedded with the sentence-transformers model all-MiniLM-L6-v2 [3] when it can be loaded. If it cannot (not installed or no network access) the system automatically uses a TF-IDF-style hashing vectoriser (1,024 dimensions, unigrams and bigrams) so that the pipeline always runs. The vectors are stored in a persistent ChromaDB collection using cosine distance. Retrieval embeds the query, returns the top-k chunks, and can filter by document type (for example playbooks only). Figure 3.2 shows the indexing and retrieval flow.

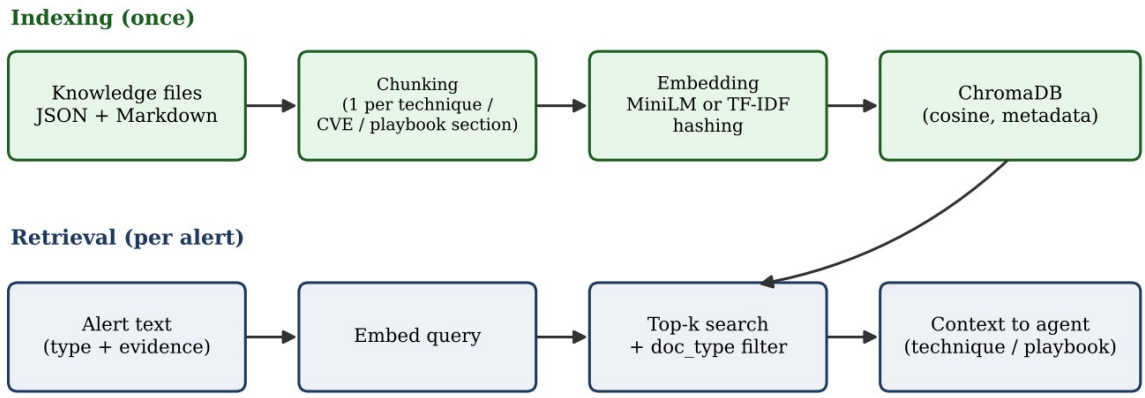


Figure 3.2: RAG indexing and retrieval pipeline

3.1.5 Multi-agent crew

The crew runs in CrewAI's sequential process: the output of each task is passed as context to the next. Table 3.4 lists the agents and their tools. The language model is accessed through CrewAI and LiteLLM [14] with a low temperature (0.1) and a request-rate limit of 20 per minute to suit

free tiers. The provider is chosen automatically from whichever API key is available (Gemini, Groq or OpenRouter).

Table 3.4: Agents, goals and tools

Agent	Goal	Tools
Log Analyst / Threat Detector	Detect malicious or anomalous events with few false positives	log_analysis, anomaly_detection
Threat Intelligence Analyst	Map each alert to ATT&CK, CVE and IOC context	threat_intel_search (RAG), ioc_lookup
Incident Responder	Contain each incident by following the retrieved playbook	playbook_search (RAG), response_action
Security Report Writer	Write an executive-ready Markdown incident report	None (uses earlier task outputs)

3.1.6 Safety controls and offline mode

- **Simulation and allow-list:** the response tool only records actions and accepts six types (block_ip, isolate_host, disable_account, reset_credentials, enable_waf_rule, create_ticket); anything else is rejected, and duplicates are skipped.
- **Playbook-driven actions:** the numbered Containment steps of the retrieved playbook are converted into actions. "Block" targets the external malicious address, "isolate" targets the internal infected host, and "disable" targets the account that logged in successfully or performed the privileged commands.
- **Low confidence is never auto-contained:** anomaly alerts only create a ticket.
- **Offline mode:** without an API key, or if the live crew fails (for example because of rate limits or a model without tool calling), the same tools and RAG run deterministically and the notebook prints the reason for the fallback.

3.2 System requirements

Table 3.5: Software, hardware and service requirements

Category	Requirement
Language and runtime	Python 3.10 or later (tested on 3.13); Jupyter Notebook or Google Colab
Core libraries	crewai, chromadb, scikit-learn, pandas, numpy, matplotlib, python-dotenv
LLM access (optional)	litellm and google-genai; a free API key for Google Gemini, Groq or OpenRouter
Embeddings (optional)	sentence-transformers (all-MiniLM-L6-v2); otherwise the built-in hashing embedder is used
Hardware	Any laptop or the free Colab tier; about 4 GB of RAM; no GPU required
Testing	pytest (25 automated tests)

3.3 Evaluation methodology

Detection is evaluated at event level: an event is predicted as an attack if it belongs to any alert, and this is compared with its ground-truth label. With TP, FP and FN denoting true positives, false positives and false negatives:

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

Classification is evaluated per attack type as the share of that attack's events covered by an alert of the correct type. An ablation compares the rule layer alone with rules plus the Isolation Forest. Retrieval is evaluated with hit-rate@k over eight natural-language queries, each with an expected ATT&CK identifier: a query is a hit if the expected technique appears in the top k results. Response correctness is checked by inspecting the generated actions against the expected containment for each incident, and 25 automated tests cover detection, retrieval, provider configuration and the offline pipeline.

CHAPTER 4
IMPLEMENTATION DETAILS AND RESULT ANALYSIS

4.1 Implementation details

The code is organised as a Python package (src/cyberthreat) with one module per concern, plus a generator script that assembles all modules and the knowledge base into one self-contained notebook that can be uploaded to Colab and run without cloning any repository. Table 4.1 lists the modules.

Table 4.1: Implementation modules

Module	Responsibility
config.py	Paths, .env and Colab-secret loading, provider auto-detection (Groq, Gemini, OpenRouter), model-name normalisation, offline fallback
data_generator.py	Benign noise and six injected, labelled attack incidents
detection.py	Alert data class, rule detectors, Isolation Forest, ATT&CK and severity maps
rag.py	Chunking, embedder (MiniLM or hashing), ChromaDB knowledge base, hit-rate metric
tools.py	Plain tool functions and CrewAI BaseTool wrappers sharing one security context
agents.py, tasks.py	Four agents and the sequential task chain
crew.py	Crew construction, live run with fallback, deterministic offline pipeline, report builder
evaluation.py	Event-level metrics, per-attack report, plots

Several practical issues were found and fixed while running the system on free hosted models, and they shaped the final design:

- CrewAI refuses to run an agent synchronously inside a running event loop, which is how Jupyter and Colab execute cells. The crew is therefore started in a worker thread.
- With the Groq model accessed through LiteLLM, CrewAI's internal prompt-cache flag was rejected by the provider's API. This showed that some provider and model combinations are incompatible with the current framework version, so any live-crew failure is caught and the offline pipeline is run instead, with the cause printed.
- The Gemini API rejected a Hugging Face style model name, and some free models do not support tool calling. Model identifiers are normalised, can be overridden with environment variables, and the notebook can list the model identifiers supported by a given key.
- The first version of the response logic blocked internal hosts and chose the wrong account to disable. Actions are now derived from a remote address recorded in each alert, and automated tests assert the correct targets.

4.2 Results/observations

All results below are from the executed notebook in offline mode with random seed 42. The log contained 2,696 events: 2,500 benign and 196 attack events. The detection layer produced eight alerts: six rule alerts (one for each injected incident) and two low-severity anomaly alerts (Figure 4.1).

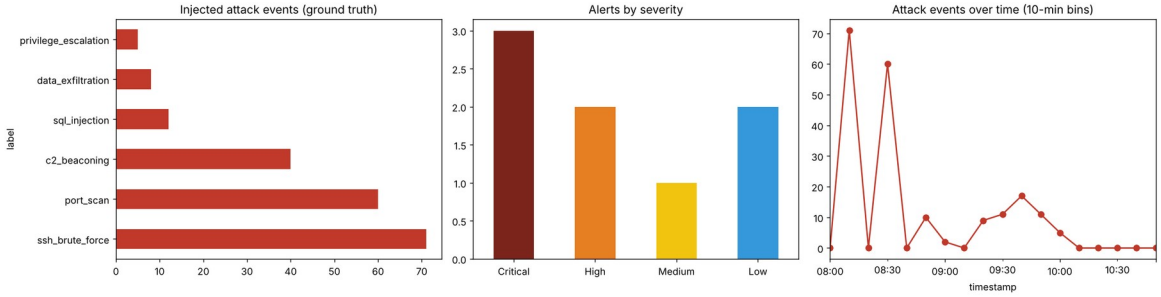


Figure 4.1: Injected attack events, alerts by severity and attack events over time

Table 4.2: Event-level detection results (seed 42)

Configuration	TP	FP	FN	TN	Precision	Recall	F1
Rules only	196	0	0	2500	1.000	1.000	1.000
Rules + Isolation Forest	196	16	0	2484	0.925	1.000	0.961

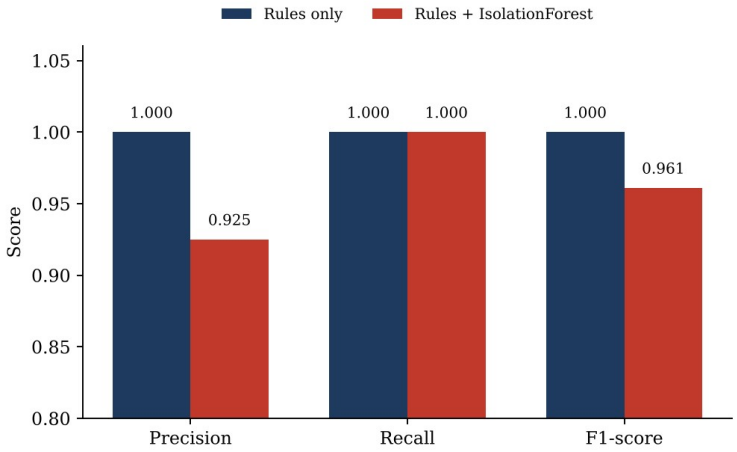


Figure 4.2: Precision, recall and F1-score of the two detector configurations

Table 4.3: Per-attack detection and classification

Attack type	Events	Detected by any alert	Correctly classified	Recall
ssh_brute_force	71	71	71	1.00
port_scan	60	60	60	1.00
sql_injection	12	12	12	1.00
c2_beaconing	40	40	40	1.00
data_exfiltration	8	8	8	1.00
privilege_escalation	5	5	5	1.00

Retrieval quality. Using the hashing embedder, the correct ATT&CK technique was the top result for 7 of the 8 test queries, giving hit-rate@1 and hit-rate@3 of 0.88. Table 4.4 shows the outcome for each query.

Table 4.4: RAG retrieval test queries (hashing embedder)

Query	Expected	Top-1 result	Outcome
many failed ssh logins from one ip	T1110	T1110	Hit
attacker scanning many ports on a server	T1046	T1071	Miss
union select in web request parameter	T1190	T1190	Hit
periodic small connections to external server at fixed interval	T1071	T1071	Hit
huge outbound data upload to unknown destination	T1041	T1041	Hit
chmod u+s and sudoers modification	T1548	T1548	Hit
login success after repeated password failures	T1110	T1110	Hit
network service discovery probing open ports	T1046	T1046	Hit

Response actions. The offline pipeline generated an incident report listing eight incidents (three Critical, two High, one Medium and two Low) and applied twelve simulated actions taken from the retrieved playbooks (Table 4.5). External attackers were blocked, infected internal hosts were isolated and compromised accounts were disabled; no internal address was blocked, and the two anomaly alerts only produced tickets.

Table 4.5: Simulated response actions generated for each incident

Incident	Severity	Actions applied
C2 beaconing (host 10.0.5.23)	Critical	isolate_host 10.0.5.23; block_ip 185.220.101.7 (C2 server)
Data exfiltration (host 10.0.3.40)	Critical	block_ip 45.133.1.88; isolate_host 10.0.3.40
Privilege escalation (user bob)	Critical	disable_account bob; isolate_host 10.0.5.14
SSH brute force (203.0.113.45)	High	block_ip 203.0.113.45; disable_account alice (the account that was logged into)
SQL injection (203.0.113.99)	High	block_ip 203.0.113.99
Port scan (198.51.100.77)	Medium	block_ip 198.51.100.77
Two anomaly alerts (10.0.5.20, 10.0.5.24)	Low	create_ticket for analyst review (no containment)

4.3 Analysis of the results

- **Detection.** The rule layer detected every injected incident without a false positive, and each was typed correctly. This is expected on a controlled benchmark because the thresholds were designed together with the generator; it validates the pipeline, not real-world accuracy.

- **Effect of the anomaly model.** Adding the Isolation Forest kept recall at 1.00 but reduced precision from 1.000 to 0.925 because it raised two Low alerts covering 16 benign events. On two other seeds (1 and 7), precision of the full system was 0.942 and 0.929 (F1 0.970 and 0.963), so the trade-off is stable. The model is useful because it can flag behaviour that no rule describes, and the cost is controlled because those alerts only create tickets.
- **Retrieval.** The single miss was the query "attacker scanning many ports on a server", which was matched to C2 beaconing. The hashing embedder compares words, and the technique text for port scanning uses different wording ("network service discovery"), so a purely lexical match is weak. A semantic embedding such as all-MiniLM-L6-v2 is designed to bridge this kind of vocabulary gap, but it was not run in the test environment, so its benefit is expected rather than measured.
- **Response correctness.** The actions followed the retrieved playbooks and targeted the right entities for every incident. Deriving the block target from the remote address in the alert, rather than from the source address, was essential: for C2 and exfiltration the source is an internal victim host that must be isolated, not blocked.
- **Reproducibility and testing.** Detection and the offline pipeline are deterministic, so the same seed gives the same alerts and report, and 25 automated tests verify detection, retrieval, provider selection, the fallback and containment targets.
- **Live LLM crew.** The live CrewAI path is implemented and its construction is unit-tested, but attempts to run it on free hosted models (Groq and Gemini) exposed the compatibility issues in Section 4.1. The quantitative results therefore come from offline mode, and the quality of LLM-written reports is not scored automatically; this is the main limitation of the evaluation.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project designed and implemented a multi-agent cyber-threat detection and response system that combines rule-based and Isolation Forest detection, a ChromaDB-backed RAG knowledge base and a four-agent CrewAI crew. On labelled synthetic telemetry the system detected all 196 attack events across six attack types and classified every incident correctly, retrieved the correct ATT&CK technique for 7 of 8 test queries even with a lightweight embedder, and produced playbook-driven, correctly targeted simulated containment actions and an incident report.

The main design lessons are that detection works best as deterministic, testable code, that retrieval keeps the agents' reasoning tied to curated knowledge, and that safety and reproducibility (simulated allow-listed actions, an offline fallback and automated tests) are as important as the use of language models.

5.2 Limitations

- The logs are synthetic and the rule thresholds were tuned alongside the generator, so performance on real traffic will differ and thresholds must be re-tuned.
- Benign periodic traffic such as monitoring agents can resemble beaconing and may cause false positives.
- The knowledge base is a small curated subset (10 techniques, 5 CVEs, 6 playbooks), not the full ATT&CK and NVD corpora.
- The quantitative results were obtained in offline mode with the hashing embedder; the live-LLM path depends on the chosen model supporting tool calling and on provider rate limits, and the quality of LLM-written reports was not scored.
- Response actions are simulated and are not connected to real firewalls or endpoint tools.

5.3 Suggestions for future directions

- Evaluate on public datasets such as CICIDS2017 [6] and on real Zeek or Suricata logs.
- Ingest the complete ATT&CK STIX data and NVD feeds, and use hybrid (keyword plus dense) retrieval with re-ranking.
- Measure the benefit of semantic embeddings and compare several LLMs for tool-calling reliability and report quality, for example with an LLM-as-judge protocol.
- Add human-in-the-loop approval and SIEM/SOAR integration for real response actions, and support streaming detection with learned thresholds.

REFERENCES

Journal / Conference Papers

[1] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation Forest”, Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), Pisa, Italy, pp. 413–422, December 2008.

[2] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), Hong Kong, pp. 3982–3992, November 2019.

[4] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models”, International Conference on Learning Representations (ICLR), 2023.

[5] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation”, arXiv preprint arXiv:2308.08155, 2023.

[6] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization”, Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116, January 2018.

Reference / Hand Books

[7] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington and C. B. Thomas, “MITRE ATT&CK: Design and Philosophy”, technical report, The MITRE Corporation, 2018.

[8] P. Cichonski, T. Millar, T. Grance and K. Scarfone, “Computer Security Incident Handling Guide”, NIST Special Publication 800-61 Revision 2, National Institute of Standards and Technology, 2012.

[9] S. Russell and P. Norvig, “Artificial Intelligence: A Modern Approach”, Pearson, 4th Edition, 2020, ISBN 978-0134610993.

Web

[10] CrewAI, CrewAI documentation, https://docs.crewai.com

[11] Chroma, ChromaDB documentation, https://docs.trychroma.com

[12] MITRE, MITRE ATT&CK knowledge base, https://attack.mitre.org

[13] scikit-learn, IsolationForest, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

[14] LiteLLM, LiteLLM documentation, https://docs.litellm.ai